

Assignment 1

Building a Reliable Mini Data Pipeline in Jupyter

Fri Aug 28 2026

Scenario

You are a junior data engineer supporting an e-commerce analytics pipeline. A CSV file containing customer events arrives from several systems. Before the data can be used by analysts, it must be validated, cleaned, transformed, and checked for performance problems.

The file `events.csv` intentionally contains realistic data-quality problems such as duplicate records, missing identifiers, malformed timestamps, inconsistent text categories, and invalid numeric values.

Learning Goals

By completing this assignment, you will practice:

- writing reusable Python functions;
- debugging errors and interpreting tracebacks in Jupyter;
- building basic data-quality checks;
- timing and profiling Python code;
- organizing a small ETL-style workflow.

Dataset

The dataset contains the following columns:

Column	Meaning
<code>event_id</code>	Unique identifier for an event
<code>event_timestamp</code>	Time at which the event occurred
<code>customer_id</code>	Customer identifier
<code>product_id</code>	Product identifier
<code>event_type</code>	view, add_to_cart, purchase, or refund
<code>quantity</code>	Number of units
<code>unit_price</code>	Price per unit
<code>country</code>	Customer country code
<code>source</code>	web, mobile, or partner_api
<code>status</code>	Processing status

Tasks

1. Build reusable cleaning functions (3 pts)

Load the CSV into a Jupyter notebook using pandas. Create and use at least **three functions**. Your functions should collectively:

- standardize `event_type` and `status` by trimming whitespace and converting text to a consistent case;
- convert `event_timestamp`, `quantity`, and `unit_price` to appropriate data types;
- remove exact duplicates and identify invalid or missing values.

For invalid numeric or timestamp values, convert them to missing values rather than allowing the entire pipeline to fail.

Rubric: 1 point for function design, 1 point for correct cleaning, and 1 point for clear function calls/docstrings or comments.

2. Debug a broken transformation (3 pts)

Start with the following intentionally broken function:

```
def calculate_revenue(df):  
    df["revenue"] = df["quantity"] * df["unit_price"]  
    purchases = df[df["event_type"] == "Purchase"]  
    return purchases.groupby("country")["revenue"].sum()
```

Run it on your cleaned data and use the traceback to debug it.

- Explain at least **two problems** in the original function.
- Correct the function.
- Show the resulting purchase revenue by country.

Use at least one Jupyter/IPython debugging feature such as `%xmode Context`, `%xmode Verbose`, or `%debug`. Include a short Markdown note describing how the traceback helped you.

Rubric: 1 point for identifying problems, 1 point for a working correction, and 1 point for demonstrating/debugging explanation.

3. Profile two implementations (2 pts)

Create two implementations that calculate revenue by country:

- Version A:** a Python loop over records or rows;
- Version B:** a pandas/vectorized solution using filtering, column arithmetic, and `groupby`.

Use `%timeit` to compare them. Then use `%prun` on at least one implementation.

Answer in 2–4 sentences:

- Which version is faster?
- What did the profiler show?
- Why does profiling matter in a data-engineering pipeline?

Rubric: 1 point for correct timing/profiling and 1 point for interpretation.

4. Produce a pipeline quality summary (2 pts)

Create a final function called:

```
def pipeline_summary(raw_df, clean_df):  
    ...
```

It should return or display at least:

- number of raw rows;
- number of cleaned rows;
- number of duplicate rows removed;
- number of records with unusable timestamps;
- number of records with missing customer or product IDs;
- total valid purchase revenue.

Add a final Markdown paragraph explaining whether you would allow this dataset to continue to a production analytics table and why.

Rubric: 1 point for the summary metrics and 1 point for the data-engineering judgment.

Submission

Submit one Jupyter notebook (`.ipynb`) containing:

- executable code;
- outputs from the major steps;
- timing/profiling output;
- short Markdown explanations;
- a final cleaned-data and pipeline-quality summary.

Your notebook should run from top to bottom without manual fixes.

Point Summary

Task	Points
Reusable cleaning functions	3
Debugging a broken transformation	3
Timing and profiling	2
Pipeline quality summary	2
Total	10